

OP2

Generated by Doxygen 1.9.4

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Human Class Reference	7
4.2 Student1 Class Reference	8
4.2.1 Member Function Documentation	9
4.2.1.1 getPavarde()	10
4.2.1.2 getVardas()	10
4.3 Vector< T > Class Template Reference	10
4.3.1 Member Function Documentation	11
4.3.1.1 clear()	11
5 File Documentation	13
5.1 header.h	13
5.2 human.hpp	13
5.3 student.hpp	14
5.4 vector.hpp	17
Index	23

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Human	7
Student1	8
Vector< T >	10

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Human	7
Student1	8
Vector< T >	10

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

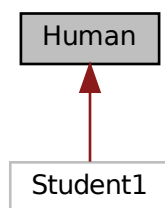
/home/xxmp/GitHub/OP2/program/ header.h	13
/home/xxmp/GitHub/OP2/program/ human.hpp	13
/home/xxmp/GitHub/OP2/program/ student.hpp	14
/home/xxmp/GitHub/OP2/program/ vector.hpp	17

Chapter 4

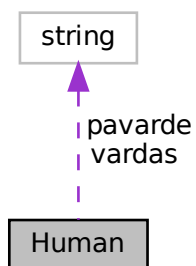
Class Documentation

4.1 Human Class Reference

Inheritance diagram for Human:



Collaboration diagram for Human:



Public Member Functions

- **Human** (const std::string &v, const std::string &p)
- **Human** & **operator=** (const **Human** &other)
- virtual void **setVardas** (std::string vardas_)=0
- virtual void **setPavarde** (std::string pavarde_)=0
- virtual std::string **getVardas** () const =0
- virtual std::string **getPavarde** () const =0

Protected Attributes

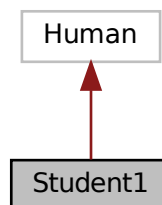
- std::string **vardas**
- std::string **pavarde**

The documentation for this class was generated from the following file:

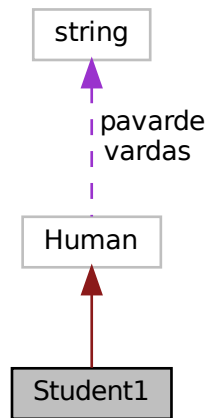
- /home/xxmp/GitHub/OP2/program/human.hpp

4.2 Student1 Class Reference

Inheritance diagram for Student1:



Collaboration diagram for Student1:



Public Member Functions

- **Student1** (string v="Default", string p="Default", int e=0, vector< int > m={0})
- **Student1** (const [Student1](#) &other)
- [Student1](#) & **operator=** (const [Student1](#) &other)
- **Student1** ([Student1](#) &&other) noexcept
- [Student1](#) & **operator=** ([Student1](#) &&other) noexcept
- bool **operator==** (const [Student1](#) &other) const
- void **setVardas** (string vardas_) override
- void **setPavarde** (string pavarde_) override
- void **setEgzaminas** (int egzaminas_)
- void **setMark** (vector< int > mark_)
- string [getVardas](#) () const override
- string [getPavarde](#) () const override
- int [getEgzaminas](#) () const
- vector< int > [getMark](#) () const
- float [getMedian](#) () const
- float [getResult](#) () const
- void **GenerateNames** ()

Friends

- std::ostream & **operator<<** (std::ostream &os, const [Student1](#) &stud)
- std::ifstream & **operator>>** (std::ifstream &in, [Student1](#) &stud)
- std::istream & **operator>>** (std::istream &is, [Student1](#) &stud)

4.2.1 Member Function Documentation

4.2.1.1 getPavarde()

```
string Student1::getPavarde ( ) const [inline], [override], [virtual]
```

Implements [Human](#).

4.2.1.2 getVardas()

```
string Student1::getVardas ( ) const [inline], [override], [virtual]
```

Implements [Human](#).

The documentation for this class was generated from the following files:

- /home/xxmp/GitHub/OP2/program/student.hpp
- /home/xxmp/GitHub/OP2/program/student.cpp

4.3 Vector< T > Class Template Reference

Public Member Functions

- **Vector** (size_t initial_capacity)
- **Vector** ([Vector](#) &&other) noexcept
- **Vector** (const [Vector](#) &other)
- **Vector** & **operator=** (const [Vector](#) &other)
- **Vector** & **operator=** ([Vector](#) &&other)
- template<typename input >
Vector (input first, input last)
- T & **at** (size_t index)
- T & **operator[]** (size_t index)
- T & **front** ()
- T & **back** ()
- T * **begin** ()
- T * **end** ()
- T * **rbegin** ()
- T * **rend** ()
- size_t **get_size** () const
- size_t **get_capacity** () const
- bool **empty** () const
- void **reserve** (size_t new_capacity)
- void **shrink_to_fit** ()
- size_t **max_size** () const
- void [clear](#) ()
- void **insert** (size_t index, const T &value)
- void **insert** (size_t index, size_t count, const T &value)
- void **push_back** (const T &value)
- void **pop_back** ()
- void **erase** (size_t index)
- T * **erase** (T *first, T *last)

Chapter 5

File Documentation

5.1 header.h

```
1 #include <algorithm>
2 #include <cctype>
3 #include <cstdio>
4 #include <cstdlib>
5 #include <iomanip>
6 #include <iostream>
7 #include <random>
8 #include <sstream>
9 #include <string>
10 #include <fstream>
11 #include <vector>
12 #include <chrono>
13 #include <filesystem>
14 #include <ios>
15 #include <stdexcept>
16
17 using std::cout;
18 using std::cin;
19 using std::endl;
20 using std::setw;
21 using std::string;
22 using std::vector;
23 using std::ifstream;
24 using std::cerr;
25 using std::left;
26 using std::right;
27
28 bool isValid ( string number );
29
30 bool isValid ( int number );
31
32 int randomNumber (int a, int b);
33
34 void readFile(vector<string>& name);
35
36 string get_name(vector <string> name);
37
38 //void count_marks (vector<Student>& stud);
39
40 vector<string> listTxtFiles();
41
42 void generate_file (int number);
43
44
```

5.2 human.hpp

```
1 #include <string>
2
3 class Human {
4     protected:
5
6     std::string vardas;
7     std::string pavarde;
```

```

8
9     public:
10
11         // Default constructor
12         Human() : vardas(""), pavarde("") {}
13
14         // Parameterized constructor
15         Human(const std::string& v, const std::string& p) : vardas(v), pavarde(p) {}
16
17
18         // virtualus destruktorius
19         virtual ~Human() {
20             //std::cout << "Virtual destructor Human was called" << std::endl;
21         }
22
23         // Copy assignment operator
24         Human& operator=(const Human& other) {
25             if (this != &other) {
26                 vardas = other.vardas;
27                 pavarde = other.pavarde;
28             }
29             return *this;
30         }
31
32         //virtualus setteriai
33         virtual void setVardas(std::string vardas_) = 0;
34
35         virtual void setPavarde(std::string pavarde_) = 0;
36
37         //virtualus getteriai
38         virtual std::string getVardas() const = 0;
39
40         virtual std::string getPavarde() const = 0;
41 };

```

5.3 student.hpp

```

1 #include <iomanip>
2 #include <ios>
3 #include <istream>
4 #include <ostream>
5 #include <fstream>
6 #include <sstream>
7 #include <string>
8 #include <vector>
9 #include <iostream>
10 #include <chrono>
11 #include <algorithm>
12 #include "../program/human.hpp"
13 #include "vector.hpp"
14
15 using std::string;
16 using std::vector;
17
18
19 class Student1 : private Human {
20
21     private:
22         //string vardas;
23         //string pavarde;
24         int egzaminas;
25         vector<int> mark;
26     public:
27
28         // konstruktorius
29         Student1(string v = "Default", string p = "Default", int e = 0, vector<int> m = {0}) :
29             egzaminas(e), mark(m) {
30             //std::cout << "Iskviestas Konstruktorius studentui " << v << std::endl;
31             vardas = v;
32             pavarde = p;
33         }
34
35         // destruktorius
36         ~Student1() {
37             //std::cout << "Iskviestas Destruktorius studentui " << vardas << std::endl;
38             mark.clear();
39         }
40
41         // copy konstruktorius
42         Student1(const Student1 &other) : Student1(other.vardas, other.pavarde, other.egzaminas,
43             other.mark) { // kodėl jei čia parašau : Student1 vardas = other.vardas....
44             Student1(const Student1 &other) { // kodėl jei čia parašau : Student1 vardas = other.vardas....
45                 vardas = other.vardas;

```

```

45     pavarde = other.pavarde;
46     egzaminas = other.egzaminas;
47     mark = other.mark;
48     //std::cout<<"Iskviestas Copy Konstruktorius studentui "<<vardas<<std::endl;
49 }
50
51 // copy assignment
52 Student1& operator=(const Student1 &other) {
53     if (this != &other) { // Patikrinti ar ne self-assigninama
54         //std::cout << "Assigning vardas, this: " << this << ", other: " << &other << std::endl;
55         vardas = other.vardas;
56         //std::cout << "Assigning pavarde" << std::endl;
57         pavarde = other.pavarde;
58         //std::cout << "Assigning egzaminas" << std::endl;
59         egzaminas = other.egzaminas;
60         //std::cout << "Assigning mark" << std::endl;
61         mark = other.mark;
62     }
63     else std::cout << "Self-assignment, skipping" <<std::endl;
64     //std::cout << "Iskviestas Copy Assignmentas studentui " << vardas <<std::endl;
65     return *this;
66 }
67
68 //move konstruktorius
69 Student1(Student1&& other) noexcept : Student1(other.vardas ,other.pavarde, other.egzaminas,
other.mark) {
70     //other = Student1();
71     //std::cout << "Iskviestas Move Konstruktorius studentui " << vardas << std::endl;
72     //}
73
74 Student1(Student1&& other) noexcept :
75 Human(std::move(other.vardas), std::move(other.pavarde)), // Move base class members
76     egzaminas(other.egzaminas),
77     mark(std::move(other.mark)) {
78     // Reset the moved-from object
79     other.vardas = "Default";
80     other.pavarde = "Default";
81     other.egzaminas = 0;
82     other.mark = {0};
83     //std::cout << "Iskviestas Move Konstruktorius studentui " << vardas << std::endl;
84 }
85
86 // move assignment
87 /*
88 Student1& operator=(Student1&& other) noexcept {
89 if (this != &other) { // Patikrinti ar ne self-moving
90 vardas = other.vardas;
91 pavarde = other.pavarde;
92 egzaminas = other.egzaminas;
93 mark = other.mark;
94
95 other = Student1();
96 }
97 else std::cout <<"Self-moving, skipping"<<std::endl;
98 //std::cout<<"Iskviestas Move Assignmentas studentui "<<vardas<<std::endl;
99 return *this;
100 }
101 */
102
103 Student1& operator=(Student1&& other) noexcept {
104     if (this != &other) { // Avoid self-move
105         // Move base class members
106         Human::operator=(std::move(other));
107
108         // Move member variables
109         egzaminas = other.egzaminas;
110         mark = std::move(other.mark);
111
112         // Reset the moved-from object
113         other.vardas = "Default";
114         other.pavarde = "Default";
115         other.egzaminas = 0;
116         other.mark = {0};
117     }
118     //std::cout<<"Iskviestas Move Assignmentas studentui "<<vardas<<std::endl;
119     return *this;
120 }
121
122 // Equality operator
123 bool operator==(const Student1& other) const {
124     return (vardas == other.vardas &&
125         pavarde == other.pavarde &&
126         egzaminas == other.egzaminas &&
127         mark == other.mark);
128 }
129
130

```

```

131 // setteriai
132
133 void setVardas (string vardas_) override;
134
135 void setPavarde(string pavarde_) override;
136
137 void setEgzaminas(int egzaminas_);
138
139 void setMark(vector<int> mark_);
140
141 // getteriai
142
143 string getVardas()const override {
144     return vardas;
145 }
146
147 string getPavarde()const override {
148     return pavarde;
149 }
150
151 int getEgzaminas()const {
152     return egzaminas;
153 }
154
155 vector<int> getMark()const {
156     return mark;
157 }
158
159 friend std::ostream& operator<<(std::ostream& os, const Student1& stud) {
160     int g = 20;
161     os << std::setw(g) << std::left << stud.getVardas()
162     << std::setw(g) << std::left << stud.getPavarde()
163     << std::setw(g) << std::left << std::setprecision(3)
164     << stud.getResult() << std::setw(g) << std::left
165     << std::setprecision(3) << stud.getMedian() << std::endl;
166     return os;
167 }
168
169 // ifstreama dar padaryt
170 friend std::ifstream& operator>>(std::ifstream& in, Student1& stud) {
171     std::string temp;
172     std::getline(in, temp); // Read the entire line
173     //std::cout << "nuskaitoma eilute \n" << std::endl;
174
175     if (temp.empty()) return in; // Skip empty lines
176
177     string random;
178     std::istringstream stream(temp); // Convert the line into a string stream
179     //std::cout << "konvertuojama i string stream \n" << std::endl;
180
181     stream >> stud.vardas >> stud.pavarde;
182
183     //std::cout << "ivesta vardas ir pavarde" << std::endl;
184     //std::cout << stud.getPavarde() << " " << stud.getPavarde() << "\n";
185
186     int marks;
187     stud.mark.clear(); // Clear marks vector
188     while (stream >> marks) {
189         stud.mark.push_back(marks);
190         //std::cout << "ivestas pazymys \n" << std::endl;
191     }
192
193     if (!stud.mark.empty()) {
194         stud.egzaminas = stud.mark.back();
195         stud.mark.pop_back();
196     } else {
197         stud.egzaminas = 0; // Default value if no marks are provided
198     }
199     //std::cout << "ivestas egzaminas \n" << std::endl;
200
201     return in;
202 }
203
204 friend std::istream& operator>>(std::istream& is, Student1& stud) {
205     std::string egzaminas, mark_temp;
206     vector<int> mark {};
207
208     std::cout << "Enter name: ";
209     is >> stud.vardas;
210
211     std::cout << "Enter surname: ";
212     is >> stud.pavarde;
213
214     while (true) {

```

```

218         std::cout << "Enter exam mark (0-10): ";
219         std::cin >> egzaminas;
220         if (std::stoi(egzaminas) >= 0 && std::stoi(egzaminas) <= 10) {
221             stud.egzaminas = std::stoi(egzaminas);
222             break;
223         }
224     }
225
226     while (true) {
227         std::cout << "Enter a mark (or 'q' to quit): ";
228         std::cin >> mark_temp;
229
230         if (mark_temp == "q") {
231             stud.mark = mark;
232             mark.clear();
233             system("clear");
234             break;
235         }
236
237         if (std::stoi(mark_temp) >= 0 && std::stoi(mark_temp) <= 10) {
238             mark.push_back(std::stoi(mark_temp));
239         }
240     }
241
242     return is;
243 }
244
245 float getMedian() const;
246
247 float getResult() const;
248
249 void GenerateNames();
250
251 };
252
253 void print(vector<Student1> &stud);
254
255 void import_file (Vector<Student1> &stud, string filename);
256
257 void sort_students (Vector<Student1>& stud);
258
259 void choose_printing_method (Vector<Student1>& stud);
260
261 void print_marks (Vector<Student1> stud);
262
263 void write_marks (Vector<Student1> stud, string name);
264
265 void divide_file1 (Vector<Student1>& stud, Vector<Student1>& kietiakai, Vector<Student1>& nuskriaustukai,
    string filename);
266
267 void divide_file2 (Vector<Student1>& stud, Vector<Student1>& nuskriaustukai, string filename);
268
269 void divide_file3 (Vector<Student1>& stud, Vector<Student1>& nuskriaustukai, string filename);
270
271 void pagrindinis_divide (Vector<Student1>& stud, Vector<Student1>& kietiakai, Vector<Student1>&
    nuskriaustukai, int num);
272
273 void pagrindinis_divide_choice(Vector<Student1>& stud);
274
275 void sort_file (Vector<Student1>& stud, string name);
276
277 void rule_of_five();
278
279 void print_metrics (string filename, float data, int num, int strategija);
280
281 int lineCount (string filename);
282
283 string extractNumbers(const std::string& str);
284
285 void compare_vectors();

```

5.4 vector.hpp

```

1 #include <cstdint>
2 #include <iostream>
3 #include <limits>
4 #include <ostream>
5
6 template <typename T>
7 class Vector {
8     private:
9         T* data;
10         size_t size;

```

```

11     size_t capacity;
12
13     void resize(size_t new_size) {
14         if (new_size <= size) {
15             return; // Tik sumazinti dydi aij bbz kas cia zodziu palikt vietoj
16         }
17         else {
18             reserve(new_size); // Rezervuoja vieta jeigu reikia
19             for (size_t i = size; i < new_size; ++i)
20                 data[i] = T(); // Naudoja default konstruktoriu naujiems objektams
21             size = new_size;
22         }
23     }
24
25 public:
26     //-----Member-funkcijos-----//
27
28     // Konstruktorius
29     Vector() : data(nullptr), size(0), capacity(0) {} //std::cout << "Iskviestas konstruktorius" <<
std::endl;
30
31     // Konstruktorius su nustatytu dydziu
32     explicit Vector(size_t initial_capacity) : data(new T[initial_capacity]), size(0),
capacity(initial_capacity) {} //std::cout << "Iskviestas konstruktorius" << std::endl;
33
34     // Move konstruktorius
35     Vector(Vector&& other) noexcept
36     : data(other.data), size(other.size), capacity(other.capacity) {
37         other.data = nullptr;
38         other.size = 0;
39         other.capacity = 0;
40     }
41
42     // Copy konstruktorius
43     Vector(const Vector& other) : size(other.size), capacity(other.capacity) {
44         data = new T[capacity];
45         for (size_t i = 0; i < size; ++i) {
46             data[i] = other.data[i];
47         }
48     }
49
50     // Destruktorius
51     ~Vector() {
52         delete[] data;
53         //std::cout << "Iskviestas destruktoriaus" << std::endl;
54     }
55
56     // Copy assignment
57     // operatorius= pvz b = a
58     Vector& operator=(const Vector& other) {
59         if (this != &other) {
60             delete[] data;
61             size = other.size;
62             capacity = other.capacity;
63             data = new T[capacity];
64             for (size_t i = 0; i < size; ++i) {
65                 data[i] = other.data[i];
66             }
67         }
68         return *this;
69     }
70
71     // Move assignment
72     // operatorius= pvz b = std::move(a)
73     Vector& operator=(Vector&& other) {
74         if (this != &other) {
75             delete[] data;
76
77             data = other.data;
78             size = other.size;
79             capacity = other.capacity;
80
81             other.data = nullptr;
82             other.size = 0;
83             other.capacity = 0;
84         }
85         return *this;
86     }
87
88     template<typename input>
89     Vector(input first, input last) {
90         size = std::distance(first, last);
91         capacity = size;
92         data = new T[capacity];

```

```

96         size_t i = 0;
97         for (auto it = first; it != last; ++it, ++i) {
98             data[i] = *it;
99         }
100     }
101
102     //-----ELEMENT-ACCESS-----//
103
104     // Pasiukti elementa su patikrinimu
105     T& at(size_t index) {
106         if (index >= size) {
107             throw std::out_of_range("Index out of range");
108         }
109         return data[index];
110     }
111
112     // Pasiukti elementa per indeksa
113     T& operator[](size_t index) {
114         return data[index];
115     }
116
117     // Paimti pirma elementa
118     T& front() {
119         return data[0];
120     }
121
122     // Pasiukti paskutini elementa
123     T& back() {
124         return data[size-1];
125     }
126
127     //-----ITERATORS-----//
128
129     // Pradzia
130     T* begin() {
131         return data;
132     }
133
134     // Pabaiga
135     T* end() {
136         return data+size;
137     }
138
139     // R Pradzia
140     T* rbegin() {
141         return data+size-1;
142     }
143
144     // R Pabaiga
145     T* rend() {
146         return data-1;
147     }
148
149     //-----CAPACITY-----//
150
151     // Gauti size
152     size_t get_size()const {
153         return size;
154     }
155
156     // Gauti capacity
157     size_t get_capacity()const {
158         return capacity;
159     }
160
161     // Patikrinti ar vektorius tuscias
162     bool empty()const {
163         if (size == 0) return true;
164         else return false;
165     }
166
167     // Rezervuoti vieta atmintyje
168     void reserve(size_t new_capacity) {
169         if (new_capacity <= capacity) return;
170         // Grow at least double to avoid frequent allocations
171         size_t new_cap = (capacity == 0) ? 1 : capacity;
172         while (new_cap < new_capacity) new_cap *= 2;
173         T* new_data = new T[new_cap];
174         for (size_t i = 0; i < size; ++i)
175             new_data[i] = data[i];
176         delete[] data;
177         data = new_data;
178         capacity = new_cap;
179     }
180
181     // Susrinkinti capacity
182     void shrink_to_fit() {

```

```

183         if (capacity > size) {
184             resize(size);
185         }
186     }
187
188     // Max dydis
189     size_t max_size() const {
190         return std::numeric_limits<size_t>::max() / sizeof(T);
191     }
192
193     //-----Modifiers-----//
194
195     // Išvalyti vektoriu
196     void clear() {
197         //delete[] data;
198         //data = nullptr;
199         size = 0;
200         //capacity = 0;
201     }
202
203     // Idėti elementą x į y vietą
204     void insert(size_t index, const T& value) {
205         if (index > size) {
206             throw std::out_of_range("Insert index out of range");
207         }
208
209         if (size == capacity) {
210             reserve(capacity == 0 ? 1 : capacity * 2);
211         }
212
213         for (size_t i = size; i > index; --i) {
214             data[i] = data[i - 1];
215             data[index] = value;
216         }
217         ++size;
218     }
219
220     // Idėti x kopiją y elemento prieš z vietą
221     void insert(size_t index, size_t count, const T& value) {
222         if (index > size) {
223             throw std::out_of_range("Insert index out of range");
224         }
225
226         if (size == capacity) {
227             reserve(capacity == 0 ? 1 : capacity * 2);
228         }
229
230         // Visus egzistuojančius elementus perkelti į dešinę
231         for (size_t i = size; i > index; --i) {
232             data[i + count - 1] = data[i - 1];
233         }
234         // Į senas vietas įdėti naujus elementus
235         for (size_t i = 0; i < count; ++i) {
236             data[index + i] = value;
237         }
238         size += count;
239     }
240
241     // Pridėti elementą į galą
242     void push_back(const T& value) {
243         if (size == capacity) {
244             reserve(capacity == 0 ? 1 : capacity * 2);
245         }
246         data[size++] = value;
247         //size++;
248     }
249
250     // Ištrinti elementą iš galo
251     void pop_back() {
252         if (size == 0) {
253             throw std::out_of_range("Index out of range");
254         }
255         data[size-1].~T(); // Iškviešti destruktorius paskutiniam elementui(?)
256         --size;
257     }
258
259     // Ištrinti elementą iš pasirinktos vietos
260     void erase(size_t index) {
261         if (index >= size) {
262             throw std::out_of_range("Index out of range");
263         }
264         for (size_t i = index; i < size - 1; ++i) {
265             data[i] = data[i+1];
266         }
267         --size;
268     }
269

```

```
270     }
271
272     // Istrinti elementu aibe is pasirinktos vietas
273     T*erase(T* first, T* last) {
274         //
275         //
276         //std::cout << "Erase from index: " << (first - data) << " to index: " << (last - data) << "
(size=" << size << ")" << std::endl;
277         //
278         // Bounds check (optional, but for debugging it's helpful)
279         if (first < data || last > data + size || first > last) {
280             throw std::out_of_range("Invalid erase range");
281         }
282
283         size_t erase_count = last - first;
284         if (erase_count == 0) return first;
285
286         // Move trailing elements forward
287         for (T* it = first; last != data + size; ++it, ++last) {
288             *it = std::move(*last);
289         }
290
291         size -= erase_count;
292         return first;
293     }
294
295     //-----//
296
297     };
```


Index

[/home/xxmp/GitHub/OP2/program/header.h](#), [13](#)
[/home/xxmp/GitHub/OP2/program/human.hpp](#), [13](#)
[/home/xxmp/GitHub/OP2/program/student.hpp](#), [14](#)
[/home/xxmp/GitHub/OP2/program/vector.hpp](#), [17](#)

clear
 [Vector< T >](#), [11](#)

getPavarde
 [Student1](#), [9](#)
getVardas
 [Student1](#), [10](#)

[Human](#), [7](#)

[Student1](#), [8](#)
 [getPavarde](#), [9](#)
 [getVardas](#), [10](#)

[Vector< T >](#), [10](#)
 [clear](#), [11](#)